

PortSwigger Web Security Academy

SQL Injection with Filter Bypass via XML Encoding

Lab status: Solved

Practice environment: PortSwigger Web Security Academy

Primary tool: Burp Suite Repeater

1. Objective

The lab demonstrates a SQL injection vulnerability in an XML-based stock-check request where a filter blocks a straightforward SQL injection string. The bypass is achieved by using XML/HTML character encoding so the application decodes the input server-side before the SQL interpreter processes it.

2. Request Structure

```
POST /product/stock HTTP/2
Content-Type: application/xml

<?xml version="1.0" encoding="UTF-8"?>
<stockCheck>
  <productId>14</productId>
  <storeId>1</storeId>
</stockCheck>
```

3. Initial Testing

The normal request returned the stock quantity successfully. A direct SQL injection attempt was detected by the lab's filter and returned **HTTP/2 403 Forbidden** with the message **"Attack detected"**.

4. Filter Bypass Technique

Instead of sending SQL metacharacters directly, the payload was XML/HTML entity encoded. For example, the encoded value represents the SQL operators and characters while remaining valid XML. The server decodes the entity values before the database query is executed.

5. Successful Payload Pattern

```
<storeId>
&#x31; UNION SELECT username || '~' || password FROM users
</storeId>
```

The successful response returned usernames and password values from the lab's users table. The extracted credentials included the administrator account, which allowed the lab to be completed.

6. Lab Completion

After obtaining the administrator credentials, the login page was used to authenticate. The browser then displayed **"Congratulations, you solved the lab!"** and the lab status changed to **Solved**.

7. Burp Suite Workflow

Step	Action
1	Open the lab and identify the XML stock-check request.
2	Send the request to Burp Suite Repeater.
3	Test the XML parameters for SQL injection behavior.
4	Observe the filter response: HTTP 403 / Attack detected.
5	Encode the SQL injection characters using XML/HTML entities.

6	Resend the modified request in Repeater.
7	Observe the returned user credentials.
8	Use the lab administrator credentials to log in and verify completion.

8. Git Notes

The lab notes and screenshots were committed locally in the Git repository. The attempted push failed because the branch name was typed as **maine** instead of **main**.

```
git add .
git commit -m "mewmew"

# Incorrect:
git push origin maine

# Correct:
git push origin main
```

9. Key Learning

A security filter that blocks raw SQL syntax can sometimes be bypassed when input is decoded or normalized at a later processing stage. Input validation should therefore be applied consistently after canonicalization, and SQL queries should use parameterized statements rather than string concatenation.

10. Final Result

Lab: SQL injection with filter bypass via XML encoding

Result: Successfully solved

Repository note: Commit created locally; push command needs main branch name.